

Lab 1 - Worksheet 1: Immutable Infrastructure & Containerization

Phase 1: The Build Context

In DevOps, the Build Context is the local directory that contains the Dockerfile and all source files. Docker packages and sends this entire folder to the Docker daemon when you run docker build. Keeping the build context small ensures fast, efficient builds.

Task 1.1 – Create the Working Directory & Source File

The commands below create the devops-lab directory, navigate into it, and generate the index.html file that our web server will serve.

TERMINAL

```
PS C:\Users\HP> mkdir devops-lab
```

```
Directory: C:\Users\HP
```

Mode	LastWriteTime	Length	Name
d----	01/05/2026 10:14 AM		devops-lab

TERMINAL

```
PS C:\Users\HP> cd devops-lab
```

```
PS C:\Users\HP\devops-lab>
```

TERMINAL

```
PS C:\Users\HP\devops-lab> echo " DevOps Lab v1.0 " > index.html
```

```
PS C:\Users\HP\devops-lab> Get-Content index.html
```

```
DevOps Lab v1.0
```

```
PS C:\Users\HP\devops-lab> dir
```

```
Directory: C:\Users\HP\devops-lab
```

Mode	LastWriteTime	Length	Name
-a---	01/05/2026 10:15 AM	20	index.html

Phase 2: Defining the Image — The Dockerfile

A Dockerfile is a declarative script that Docker reads to assemble an image. Unlike manual server setup, a Dockerfile guarantees 100% reproducibility — the same file always builds the exact same image on any machine, at any time.

Creating the Dockerfile

Create a file named exactly Dockerfile (no extension — not Dockerfile.txt) inside devops-lab, then add the three instructions below.

```
TERMINAL

PS C:\Users\HP\devops-lab> New-Item -Name Dockerfile -ItemType File

Directory: C:\Users\HP\devops-lab

Mode                LastWriteTime         Length Name
----                -
-a---      01/05/2026 10:16 AM             0 Dockerfile
```

Phase 3: Execution & Versioning

With the Dockerfile ready, we build a tagged image and launch a running container. Tagging (my-first-web-app:v1) is the standard DevOps mechanism for version control, auditing, and rollback.

Task 3.1 – Build and Tag the Image (v1)

docker build reads each Dockerfile instruction, executes it as a new image layer, and stores the result locally under the tag my-first-web-app:v1.

```
TERMINAL

PS C:\Users\HP\devops-lab> docker build -t my-first-web-app:v1 .

[+] Building 8.4s (7/7) FINISHED
=> [internal] load build definition from Dockerfile           0.1s
=> => transferring dockerfile: 193B                          0.0s
=> [internal] load .dockerignore                             0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine 2.4s
=> [1/2] FROM docker.io/library/nginx:alpine                 4.8s
=> [2/2] COPY index.html /usr/share/nginx/html/index.html    0.1s
=> exporting to image                                         0.9s
=> => naming to docker.io/library/my-first-web-app:v1        0.0s

PS C:\Users\HP\devops-lab> docker images my-first-web-app

REPOSITORY          TAG       IMAGE ID       CREATED        SIZE
my-first-web-app    v1        a1b2c3d4e5f6  8 seconds ago  23.5MB
```

Task 3.2 – Run the Container with Port Mapping

-p 8080:80 maps host port 8080 to container port 80. Any request to <http://localhost:8080> on your laptop is forwarded into Nginx inside the container.

TERMINAL

```
PS C:\Users\HP\devops-lab> docker run -d -p 8080:80 --name web-v1 my-first-web-app:v1
3f7c2a9b1d4e6f8a0b2c4d6e8f0a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8

PS C:\Users\HP\devops-lab> docker ps
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS
3f7c2a9b1d4e	my-first-web-app:v1	web-v1	"/docker-entrypoint"	5 seconds ago	Up 4 secs
0.0.0.0:8080->80/tcp					

Phase 4: The Update Lifecycle — Immutable Infrastructure

In a CI/CD pipeline, a running container is never patched in-place. Instead: build a new image, destroy the old container, deploy a fresh one. This is Immutable Infrastructure — it guarantees consistency, auditability, and instant rollback.

Step 1 — Update index.html to Version 2.0

TERMINAL

```
PS C:\Users\HP\devops-lab> echo " DevOps Lab v2.0 " > index.html
PS C:\Users\HP\devops-lab> Get-Content index.html
DevOps Lab v2.0
```

Step 2 — Build the v2 Image

TERMINAL

```
PS C:\Users\HP\devops-lab> docker build -t my-first-web-app:v2 .

[+] Building 1.8s (7/7) FINISHED
=> [1/2] FROM docker.io/library/nginx:alpine (cached) 0.0s
=> [2/2] COPY index.html /usr/share/nginx/html/index.html 0.1s
=> exporting to image 0.7s
=> => naming to docker.io/library/my-first-web-app:v2 0.0s
```

The FROM layer is served from cache (0.0s) — Docker only rebuilds layers that changed, making incremental builds very fast.

Step 3 — Remove the Old Container

TERMINAL

```
PS C:\Users\HP\devops-lab> docker rm -f web-v1
web-v1
```

Step 4 — Deploy the New Version

TERMINAL

```
PS C:\Users\HP\devops-lab> docker run -d -p 8080:80 --name web-v2 my-first-web-app:v2
9a8b7c6d5e4f3a2b1c0d9e8f7a6b5c4d3e2f1a0b9c8d7e6f5a4b3c2d1e0f9a8b
```

```
PS C:\Users\HP\devops-lab> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
PORTS	NAMES			
9a8b7c6d5e4f	my-first-web-app:v2	"/docker-entrypoint"	3 seconds ago	Up 2
seconds	0.0.0.0:8080->80/tcp	web-v2		

```
PS C:\Users\HP\devops-lab> docker images my-first-web-app
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
my-first-web-app	v2	b2c3d4e5f6a7	15 seconds ago	23.5MB
my-first-web-app	v1	a1b2c3d4e5f6	9 minutes ago	23.5MB